

GIT

GIT

Présentation de GIT

- Pourquoi utiliser un outil de gestion de versions
- L'histoire
- Les concepts clés
- Ressources

Côté client

- Installation
- Les commandes GIT
 - Avec un dépôt local
 - Revenir en arrière
 - Avec des branches
 - Avec un serveur
- Les clients avec interface graphique

Côté serveur

- Les fournisseurs en ligne
- Installer son propre serveur
 - Git
 - Gogs
 - Gitlab

Usage avancé

- Workflow
 - Gitflow
 - Github flow
- Revue de code
- Intégration continue
- GitOps
- Les commandes avancées

Présentation de GIT

Pourquoi utiliser un outil de gestion de versions

Git est un logiciel Open Source de gestion de versions décentralisé.

Il permet de créer des versions de fichier et de les envoyer sur un serveur.

Intérêts :

- Sauvegarde et suivi
- Collaboration
- Base pour les outils de déploiement

L'histoire

Git est développé à partir de 2005 par Linus Torvalds pour le noyau Linux.

OpenSource, Très Rapide, non linéaire, adapté au gros projet

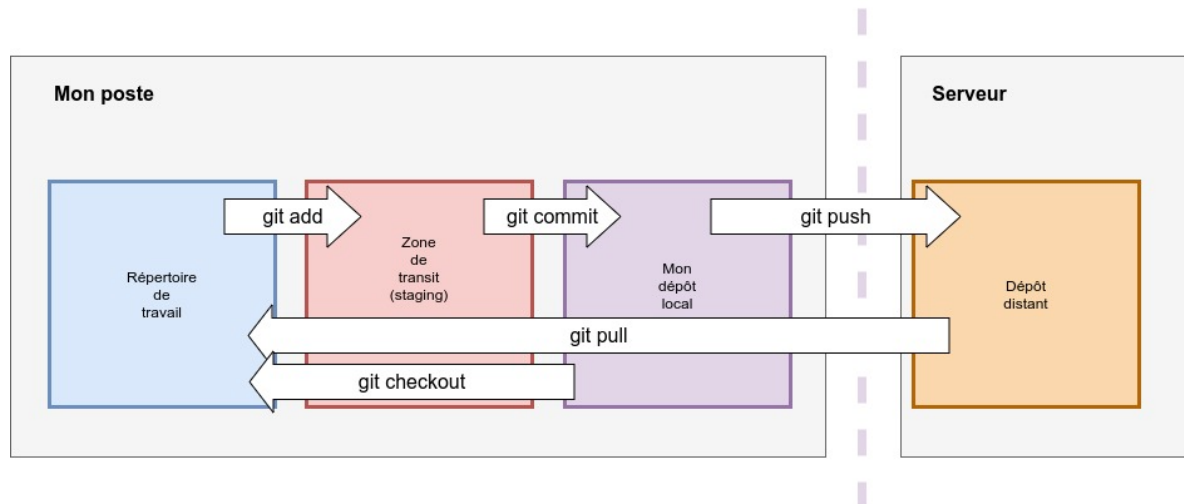
Le plus largement utilisé aujourd'hui

Les concepts clés

Un dépôt GIT : dossier qui contient les fichiers sources et suit les modifications

Le "commit" : action d'enregistrement des modifications avec un message associé

Les branches : gestion non linéaire, permet à chacun de créer des versions



Ressources

Documentation officielle

<http://git-scm.com/>

Aide en ligne de commande

```
git help [-g]
```

EBook gratuit

<https://git-scm.com/book/fr/v2>

Démo live

<https://pcottle.github.io/learnGitBranching/?demo>

Aide mémoire

Assez complet en français

<https://gist.github.com/aquelito/8596717>

Résumé (Anglais)

<https://gitsheet.wtf/>

Côté client

Installation

Sous Linux

```
sudo apt install git
```

Sous mac

```
brew install git
```

Sous Windows

<http://git-scm.com/downloads>

<https://github.com/git-for-windows/git/releases/tag/v2.32.0.windows.1>

ou embarqué avec le client Github :

<https://desktop.github.com/>

Les commandes GIT

Avec un dépôt local

Afficher l'aide

```
git help
```

Initialisation d'un dépôt

```
git init
```

Afficher l'état du dépôt local

```
git status
```

Ajouter un fichier à l'index

```
git add monfichier.txt
```

Annuler les modifications dans le répertoire de travail

```
git restore monfichier.txt
```

Valider les modifications

```
git commit -m "le commentaire"
```

Ignorer les modifications de certains fichiers

```
echo ".DS_Store" > .gitignore
```

Exercice

Initialiser un dépôt Git "monprojet", créer un fichier "test.txt" et l'ajouter à l'index.

Supprimé le fichier "test.txt" de votre poste et récupéré le avec les commandes GIT.

Revenir en arrière

Afficher l'historique

```
git log
```

Voir les différences

```
git diff [--cached]
```

`--cached` permet de voir les modifications indexées

Voir les différences avec un commit

```
git diff [hash du commit]
```

Revenir à un commit (Attention, réécriture de l'historique GIT, à n'utiliser que sur ses branches)

```
git reset [hash du commit]
```

Exercice

Mettre du texte dans un fichier, l'indexé et valider la modification.

Supprimer du texte, et valider.

Tenter de récupérer le texte avec les commandes GIT

Avec des branches

Lister les branches (qui ont déjà des commits)

```
git branch -a
```

Créer une branche et basculer dessus :

```
git checkout -b "dev"
```

Basculer avec `switch`:

```
git switch master
```

Créer une branche avec `switch`:

```
git switch -c mabranche
```

Fusionner deux branches

```
git switch master  
git merge dev
```

Suppression d'une branche

```
git branch -d mabranche
```

Remiser des modifications

```
git stash
```

Sortir de la remise

```
git stash pop
```

Exercice

Valider des modifications sur une branche "dev" et les répercuter sur "master".

Avec un serveur

Récupérer un dépôt distant

```
git clone
```

Mettre à jour le dépôt local à partir du dépôt distant (Tirer)

```
git pull
```

Mettre à jour le dépôt distant à partir du dépôt local (Pousser)

```
git push
```

Mettre uniquement les infos du dépôt à jour

```
git fetch
```

Restaurer le dépôt tel qu'il l'était lors du commit spécifié (Attention: modification du répertoire courant, potentiel perte de travail)

```
git revert [commit]
```

Gestion des conflits

Lors d'un merge, si un fichier est modifié des deux côtés, une erreur s'affiche :

```
CONFLIT (ajout/ajout) : Conflit de fusion dans main.py
Fusion automatique de main.py
La fusion automatique a échoué ; réglez les conflits et validez le résultat.
```

La commande `git status` révèle le fichier en conflit :

```
Sur la branche dev
Vous avez des chemins non fusionnés.
  (réglez les conflits puis lancez "git commit")
  (utilisez "git merge --abort" pour annuler la fusion)

Chemins non fusionnés :
  (utilisez "git add <fichier>..." pour marquer comme résolu)
  ajouté de deux côtés :    main.py

aucune modification n'a été ajoutée à la validation (utilisez "git add" ou
"git commit -a")
```

Le conflit apparaît dans le fichier, `cat main.py`

```
<<<<<< HEAD
test premiere ligne
=====
fix2 aussi sur 1ere ligne
>>>>>> fixture2
```

Je garde le contenu de la branche courante marquée par "HEAD" (tête de la branche), `cat main.py` :

```
test premiere ligne
```

ou, le contenu de la branche ("`>>>>>> [nom de la branche]`"), `cat main.py` :

```
fix2 aussi sur 1ere ligne
```

Une fois le conflit résolu ("suppression des `<<`, `==` et `>>`"), je valide les modifications liées à la fusion :

```
git add main.py
git commit -m "merge fixture2 to dev"
```

Identifier les personnes qui ont modifié un fichier

```
git blame [fichier]
```

Pousser une branche locale sur le dépôt distant

```
git push --set-upstream origin mabranche
```

Supprimer une branche sur le dépôt distant

```
git push origin --delete mabranche
```

Exercice

Pré-requis : Installer le client GIT

Dépôt distant : <http://212.47.231.145/AIS/collaborate.git>

Sur un dépôt commun, ajout du texte sur un même fichier (2 personnes par fichier) dans des branches différentes et fusionner les branches (voir dans quel cas il y a conflit et tenter de les résoudre)

En HTTP :

```
git clone http://212.47.231.145/AIS/collaborate.git
```

En SSH :

```
git clone git@212.47.231.145:AIS/collaborate.git
```

Les clients avec interface graphique

Présentation rapide de Tortoise Git / Sourcetree / Github Desktop / GitKraken / Git Cola / Intégré à l'IDE comme VS Code / Ungit

Côté serveur

Les fournisseurs en ligne

Présentation rapide de Github / Bitbucket (<https://bitbucket.org/>) / Gitlab

Installer son propre serveur

Git

Installation d'un serveur Git en ligne de commande

<https://git-scm.com/book/fr/v2/Git-sur-le-serveur-Mise-en-place-du-serveur>

Gogs

Déploiement d'un serveur avec une interface Web

<https://gogs.io/docs/installation>

Gitlab

Présentation du produit

Usage avancé

Workflow

Gitflow

<https://jeffkreeftmeijer.com/git-flow/>

Github flow

<https://guides.github.com/introduction/flow/>

Revue de code

Gerrit > <https://www.gerritcodereview.com/>

Review Board / Open Source / <https://www.reviewboard.org/>

Crucible d'Atlassian.

Intégration continue

GitOps

Les commandes avancées

git tag

rebasage

avance rapide

amend

cherry pick

Cheatsheet GIT

<https://www.atlassian.com/fr/git/tutorials/atlassian-git-cheatsheet>